

RDS & Aurora PostgreSQL

Cluster Architecture · Serverless · Proxy · PITR · Migrations

AWS Tutorial · ShopFlow

06-00

Chapter Overview — RDS & Aurora PostgreSQL

ShopFlow runs Aurora PostgreSQL 15 as its primary OLTP database: 1M MAU, 10K req/s Black Friday peak. Aurora's shared storage, 15-second failover, and cluster endpoint routing make it the correct choice over RDS PostgreSQL for production e-commerce at this scale.

Topic	Service	ShopFlow Config	Key Number
Engine	Aurora PG 15	db.r6g.large writer + 2 readers (Graviton2)	5x throughput vs RDS
Storage	Distributed shared	6 copies / 3 AZs, auto-grow 10GB increments	Max 128 TB
Failover	Promote reader	No data sync needed — shared storage	15-30 seconds
Connection pool	HikariCP	maxPoolSize=20, timeout=30s per ECS task	400 conns normal
Lambda pool	RDS Proxy	Multiplex to 50 pooled connections	1,000 Lambdas → 50 conns
Serverless v2	ACU auto-scale	0.5–8 ACU for dev/staging	85% cheaper vs provisioned
Query analysis	Performance Insights	Top SQL weekly, EXPLAIN ANALYZE monthly	N+1: 21q → 1q (22x)
Security	VPC+KMS+pgaudit	PRIVATE_ISOLATED, force_ssl=1, IAM DB auth	PCI DSS compliant
PITR	WAL stream to S3	35-day retention, 1-second RPO	RTO: 45 minutes
Schema	PG 15 features	UUID PK, JSONB+GIN, BRIN on time-series	BRIN: 128 vs 4K pages
Migrations	Flyway	Expand-contract: zero-downtime on 50M rows	4 metadata-only steps
Global DB	Aurora Global	Phase 2: eu-west-1 reader, <1s replication	40ms EU latency
Cost	1yr Reserved Instances	RI on all 3 instances in cluster	\$225/month saved (45%)

■ ShopFlow runs Java 21 on Amazon Linux 2023 ARM (Graviton2) for ECS Fargate tasks and db.r6g (Graviton2) for Aurora. Consistent ARM ISA: 12% cheaper than x86 r6 for the same vCPU/RAM, deterministic latency, and no cross-ISA JIT compilation differences between dev (ARM laptops) and production.

06-01

RDS PostgreSQL vs Aurora PostgreSQL — Architecture Decision

The fundamental difference: RDS uses one EBS volume per instance with synchronous binlog replication to standby. Aurora separates storage from compute — all instances share one 6-copy distributed volume. No binlog means no lag between writer and readers and 4x faster failover.

Feature	RDS PostgreSQL 15	Aurora PostgreSQL 15	Decision
Storage model	EBS gp3 per instance + binlog sync standby	Shared distributed — 6 copies, 3 AZs	Aurora: no binlog overhead
Throughput	1x baseline (EBS I/O bound)	5x baseline (storage-level optimization)	Aurora 5x faster
Failover time	60-120s (EBS volume promote + DNS TTL)	15-30s (no data sync — already shared)	Aurora 4x faster
Max read replicas	5 cross-region total	15 in-cluster + 5 cross-region = 20	Aurora: 4x more
Reader endpoint	None — individual instance DNS only	Cluster reader — auto load-balances	Aurora: operational simplicity
Auto-scale storage	Manual resize with brief I/O pause	10GB auto-increments, zero downtime, 128TB max	Aurora: zero ops

Feature	RDS PostgreSQL 15	Aurora PostgreSQL 15	Decision
Aurora Global DB	Not available	<1s cross-region via dedicated storage Amazon S3	Aurora exclusive feature
Backtrack (rewind)	Not available	Aurora MySQL only — NOT Aurora PostgreSQL	Use SQLTR for Aurora PG
Cost per instance	Baseline	~20% higher per instance-hour	RDS only if <500 req/s, stable

Why ShopFlow Chose Aurora PostgreSQL 15

- 10,000 req/s Black Friday peak — throughput critical
 - RDS PG EBS I/O saturates at ~4,000 req/s
 - Aurora 5x handles 10,000+ req/s without storage saturation
- 99.99% SLA — checkout flow cannot be down 60-120 seconds
 - RDS failover 60-120s = visible checkout errors during failover
 - Aurora failover 15-30s = brief retry, no visible outage to users
- No DBA team — zero-ops storage management required
 - Aurora: auto-grows, 6 copies, auto-backups by default

■ Aurora write quorum: 4/6 storage copies must confirm before write acknowledged. Can lose 2 copies and still write, lose 3 and still read. This 6-copy durability is the DEFAULT on every Aurora cluster — not an option to enable. There is no "Multi-AZ" checkbox for Aurora (it is always multi-AZ).

06-02

Aurora Cluster Architecture — Endpoints & Topology

Aurora provides four endpoint types for different access patterns. ShopFlow uses cluster endpoint (writes) and reader endpoint (reads) exclusively. Spring Boot's @Transactional(readOnly=true) annotation routes 60% of queries to the reader pool transparently.

Endpoint Type	Resolves To	Updates On Failover?	ShopFlow Use
Cluster endpoint	Current writer instance	Yes — DNS update in <30s	spring.datasource.url (all writes)
Reader endpoint	Round-robin across healthy readers	Yes — removes failed reader	datasource-replica.url (all reads)
Instance endpoint	Specific instance DNS (never changes)	No — instance-specific	DBA access, pinned analytics
Custom endpoint	Named subset of instances	Manual management only	reader-2 custom for BI workloads

ShopFlow Aurora Cluster — 3 AZ Production Topology

us-east-1a: db.r6g.large WRITER

Cluster endpoint resolves here — all @Transactional writes

Performance Insights enabled — Top SQL visible in AWS console

us-east-1b: db.r6g.large READER-1 (promotionTier=1)

SAME instance class as writer — correct sizing after failover

Reader endpoint routes @Transactional(readOnly=true) traffic here

us-east-1c: db.r6g.medium READER-2 (promotionTier=2)

Custom endpoint for analytics, BI dashboards, batch reports

Shared storage: 6 copies across 3 AZs | write quorum 4/6

Aurora CDK — Production Cluster

```
const db = new rds.DatabaseCluster(this, "ShopFlowDB", {
  engine: rds.DatabaseClusterEngine.auroraPostgres({
    version: rds.AuroraPostgresEngineVersion.VER_15_4 }),
  writer: rds.ClusterInstance.provisioned("writer", {
    instanceType: ec2.InstanceType.of(ec2.InstanceClass.R6G, ec2.InstanceSize.LARGE),
    enablePerformanceInsights: true }),
  readers: [
    rds.ClusterInstance.provisioned("r1", { // tier-1 = first failover
      instanceType: ec2.InstanceType.of(ec2.InstanceClass.R6G, ec2.InstanceSize.LARGE),
      promotionTier: 1, enablePerformanceInsights: true }),
    rds.ClusterInstance.provisioned("r2", { // tier-2 = analytics
      instanceType: ec2.InstanceType.of(ec2.InstanceClass.R6G, ec2.InstanceSize.MEDIUM),
      promotionTier: 2 }),
  ],
  vpc: vpcSubnets: { subnetType: ec2.SubnetType.PRIVATE_ISOLATED },
  securityGroups: [sgRds], storageEncrypted: true,
  backup: { retention: Duration.days(35) }, deletionProtection: true,
  parameterGroup: clusterParamGroup,
});
```

■ Always set promotionTier:1 on the same instance class as the writer. Aurora promotes the lowest tier number first. If writer is db.r6g.large and only reader-1 is db.r6g.medium at tier 1, the post-failover writer is undersized — cannot sustain production write load until manual resize (30+ minutes of degraded performance).

06-03

Spring Boot DataSource Routing — application.yml

Two HikariCP DataSources connect independently to writer (cluster endpoint) and reader (reader endpoint). Spring routes @Transactional vs @Transactional(readOnly=true) transparently via AbstractRoutingDataSource. Connection count planning prevents cascading FATAL errors during ECS scale-out.

Dual DataSource Configuration

```
spring:
  datasource: # WRITER
    url: "jdbc:postgresql://${DB_WRITER}:5432/shopflow
    ?sslmode=require&connectTimeout;=5&socketTimeout;=60"
    username: shopflow_app
    password: "${DB_PASSWORD}" # ECS secret ref to Secrets Manager
```

```
hikari:
maximum-pool-size: 20
minimum-idle: 5
connection-timeout: 30000 # ms — max wait for pool connection
idle-timeout: 600000 # 10m — close idle connections
max-lifetime: 1800000 # 30m — recycle < Aurora wait_timeout
leak-detection-threshold: 10000
connection-test-query: "SELECT 1"
datasource-replica: # READER
url: "jdbc:postgresql://${DB_READER}:5432/shopflow
?sslmode=require&readOnly=true"
hikari:
maximum-pool-size: 30 # more readers = larger pool acceptable
minimum-idle: 3
```

Connection Count Planning

Service	Normal Tasks	Peak Tasks	Pool/task	Normal Conns	Peak Conns
product-svc	4	8	20	80	160
order-svc	4	8	20	80	160
user-svc	3	5	20	60	100
inventory-svc	3	5	20	60	100
notification-svc	2	3	20	40	60
Lambda via RDS Proxy		—	50 pooled	50	50
TOTAL (% of 1,717 max)		—	—	370 (22%)	630 (37%)

■ max_connections formula: LEAST({DBInstanceClassMemory/9531392}, 5000). For r6g.large (16,384 MB): LEAST(1,762, 5000) = 1,762 (Aurora uses 1,717 after internal reservations). Alert at 80% = 1,374. Re-run this math before adding any new service or scaling existing services during Black Friday planning.

06-04 HikariCP Parameters & Pool Sizing Rules

HikariCP maintains pre-established TCP+TLS connections, eliminating 50-200ms handshake latency per query. Each ECS task has its own independent pool. Configuration errors are the top cause of production database outages for Java Spring Boot applications.

Parameter	ShopFlow Value	Purpose	Wrong Value Impact
maximumPoolSize	20	Max connections per task pool	Too high: exceed max_connections. Too low: ...
minimumIdle	5	Pre-warmed idle connections kept ready	Too low: cold starts on traffic bursts. Cost: ...
connectionTimeout	30,000ms	Max wait to acquire connection from pool	Too low (1s): false exceptions at minor bursts
idleTimeout	600,000ms	Close idle connections after 10 minutes	Too short: connection churn. Too long: stale ...
maxLifetime	1,800,000ms	Force recycle connection after 30 minutes	Must be < Aurora wait_timeout (28,800s)
leakDetectionThreshold	10,000ms	Warn if connection held longer than 10s	Unset: silent leaks exhaust pool over hours
connectionTestQuery	SELECT 1	Validate connection before borrow from pool	Unset: stale connections fail on the first read

Pool Sizing Math — Black Friday Scenario

Scenario	Calculation	Result	Safe?
Normal production (16 ECS tasks)	16 tasks × pool 20 + 50 proxy	370 / 1,717 max = 22%	Yes — 4x headroom
Black Friday peak (29 tasks)	29 tasks × pool 20 + 50 proxy	630 / 1,717 max = 37%	Yes — 2.7x headroom

Scenario	Calculation	Result	Safe?
Naive config (pool=100)	16 tasks × pool 100	1,600 / 1,717 = 93%	No — fatal at any surge
ECS deploy overlap (32 tasks)	32 tasks × pool 20 + 50	690 / 1,717 = 40%	Yes — rolling deploy safe

Java: Monitor HikariCP Pool at Runtime

```
@Scheduled(fixedRate = 60_000)
public void logPoolStats() {
    var pool = ((HikariDataSource) ds).getHikariPoolMXBean();
    log.info("HikariCP: active={} idle={} waiting={} total={}",
        pool.getActiveConnections(), pool.getIdleConnections(),
        pool.getThreadsAwaitingConnection(), pool.getTotalConnections());
    if (pool.getThreadsAwaitingConnection() > 5)
        alerts.fire("Pool contention: " + pool.getThreadsAwaitingConnection());
}
```

■ Never set maximumPoolSize=100 "for safety". At 5 services × 4 tasks × pool 100 = 2,000 connections. Aurora r6g.large max = 1,717. During ECS rolling deployments old + new tasks run simultaneously: 5 × 8 × 100 = 4,000 connections — random FATAL connection errors for 30+ minutes. The math must hold during both normal and deploy-time task counts.

06-05 HikariCP Mistakes & Leak Detection

The five most common HikariCP misconfigurations in production Java microservices. All cause silent failures — the application appears healthy until connection pool exhaustion or stale connection errors surface hours or days later under load.

Mistake	Symptom	Root Cause	Correct Fix
maximumPoolSize too large	FATAL: connection slots exhausted	total_tasks * pool_size > max_connections	max_connections ≤ 80% max_connections at peak
No leakDetectionThreshold	Pool exhaustion hours after deploy	Connections never returned to pool	Set to 10,000ms
maxLifetime > wait_timeout	Broken pipe / I/O error exactly every 8 hours	Idle conn, pool size increased	Set to 1,800,000ms (30min << wait_timeout)
readOnly not set on replica	Writes hit reader endpoint — silent failure	Aurora reader accepts writes	Adjusted on some configs
sslmode not set in JDBC URL	Plaintext credentials in transit — default JDBC uses no SSL	Default JDBC uses no SSL	Always: sslmode=require in every JDBC conn

Finding a Connection Leak — Stack Trace from leakDetectionThreshold

```
# When connection held > 10s, HikariCP logs:
WARN c.z.h.p.ProxyLeakTask - Connection leak detection triggered for
HikariPool-1 on thread http-nio-8080-exec-3,
stack trace follows:
at c.shopflow.product.ProductRepository.findByIdByCategory(ProductRepository.java:47)
at c.shopflow.product.ProductService.getCatalog(ProductService.java:123)
at c.shopflow.product.ProductController.list(ProductController.java:67)
# Root cause: ResultSet opened but not closed in finally block
# Fix: use try-with-resources or Spring @Transactional to auto-close:
@Transactional(readOnly = true)
public List<Product> getCatalog(UUID categoryId) {
    return productRepo.findByIdByCategoryId(categoryId); // auto-closes
}
```

HikariCP Health Indicator — Spring Actuator

```
# application.yml
management.health.db.enabled: true
management.endpoint.health.show-details: always
# GET /actuator/health → shows hikariPool status
# {"status":"UP", "components":{"db":{"status":"UP",
# "details":{"database":"PostgreSQL", "validationQuery":"SELECT 1"}}}}
```

■ Add leakDetectionThreshold on day one — not when you first see pool exhaustion. Connection leaks are silent: the app looks healthy until the pool is completely exhausted, then every request fails with "Connection is not available, request timed out after 30000ms." The stack trace from leak detection points to the exact line number of the unclosed connection.

06-06

Aurora Serverless v2 — Dev & Staging Environments

Aurora Serverless v2 scales between 0.5 and 128 ACU (Aurora Capacity Units) in response to actual database load. ShopFlow uses Serverless v2 for dev and staging: 85% cost reduction versus provisioned db.r6g.large. Production always uses provisioned for predictable sub-millisecond latency.

ACU	RAM	Cost/hr	ShopFlow Env	Notes
0.5	~1 GB	\$0.06	Dev: off-hours minimum	Cannot scale to zero. Always-on = ~\$22/month.
2	~4 GB	\$0.24	Dev: active development	Scales in seconds (sub-minute warmup).
4	~8 GB	\$0.48	Staging: integration tests	200+ concurrent test connections.
8	~16 GB	\$0.96	Staging: perf tests	Matches prod r6g.large RAM exactly.

Serverless v2 CDK — Dev Cluster

```
const devDb = new rds.DatabaseCluster(this, "ShopFlowDevDB", {
  engine: rds.DatabaseClusterEngine.auroraPostgres({
    version: rds.AuroraPostgresEngineVersion.VER_15_4 }),
  writer: rds.ClusterInstance.serverlessV2("writer"),
  serverlessV2MinCapacity: 0.5, // always-on minimum — $22/month idle
  serverlessV2MaxCapacity: 8, // matches prod r6g.large RAM (16 GB)
  vpc, deletionProtection: false,
  backup: { retention: Duration.days(1) },
  removalPolicy: RemovalPolicy.DESTROY,
});
```

Cost Comparison — Provisioned vs Serverless v2

Environment	Provisioned r6g.large/mo	Serverless v2/mo	Monthly Saving	Use Case
dev	\$202 (On-Demand 24/7)	\$28 (2 ACU peak, idle over 1yr)	\$174 (86%)	Individual developer clusters
staging	\$202	\$42 (4 ACU peak, batch tests)	\$160 (79%)	CI integration tests
production	\$112 (1-yr RI)	N/A — use provisioned	—	Consistent <2ms latency required

Serverless v2 vs Provisioned — Decision Matrix

Criterion	Serverless v2	Provisioned
Traffic pattern	Bursty, idle periods (dev/staging/batch)	Steady-state >30% utilization (production)
Latency SLA	2-5s scale-up lag acceptable	Sub-second latency required always
Reserved Instance discounts	Not eligible	40-45% with 1-year commitment
Scale-to-zero capability	Cannot — minimum 0.5 ACU (\$22/month)	N/A — fixed instance size

■ Serverless v2 scale-up happens in seconds. When idle at 0.5 ACU and a batch job starts, Aurora scales from 0.5 to 4 ACU in approximately 15-30 seconds. During scale-up, query latency increases. For dev: acceptable. For production: unacceptable during morning traffic surge. Always use provisioned in production.

06-07

RDS Proxy — Solving Lambda Connection Exhaustion

Lambda functions open one new database connection per invocation — there is no persistent connection pool across invocations. At 1,000 concurrent Lambda invocations, ShopFlow would create 1,000 direct DB connections (58% of

max_connections). RDS Proxy multiplexes these into 50 stable pool connections.

Lambda Connection Problem: Without vs With RDS Proxy

WITHOUT PROXY: Each Lambda invocation opens new TCP+TLS connection

500 Lambdas = 500 connections + 370 ECS = 870 total (51% max)

1,000 Lambdas + ECS = 1,370 total (80% max) — critical threshold

Black Friday 2,000 Lambdas = 2,370 total — FATAL: exceeds max_connections

WITH PROXY: Lambda connections multiplexed by RDS Proxy process

Proxy maintains 50 stable connections to Aurora cluster

1,000 Lambdas → Proxy → 50 DB connections = 420 total (24% max)

Proxy also handles failover: re-routes in 30s, Lambda caller retries once

RDS Proxy CDK

```
const proxy = new rds.DatabaseProxy(this, "ShopFlowProxy", {
  proxyTarget: rds.ProxyTarget.fromCluster(dbCluster),
  vpc, requireTLS: true, iamAuth: true,
  maxConnectionsPercent: 70, // proxy holds up to 70% of max_connections
  maxIdleConnectionsPercent: 50,
  connectionBorrowTimeout: Duration.seconds(30),
  securityGroups: [sgProxy], // sg-rds must allow from sg-proxy
});
// Grant Lambda role permission to connect via proxy
proxy.grantConnect(orderLambdaFn, "shopflow_lambda");
// Lambda env var: use proxy endpoint, NOT cluster endpoint
// DB_HOST = proxy-xxxx.proxy-xxxx.us-east-1.rds.amazonaws.com
```

Metric	Without Proxy	With Proxy	Verdict
1,000 Lambda concurrency	1,370 total conns (80% max)	420 total conns (24% max)	Must use proxy
Failover transparency	Lambda gets TCP error; must implement retry	Proxy re-routes in 30s; single retry	Proxy efficient
IAM token caching	Lambda caches per-instance (may expire)	Proxy caches IAM tokens centrally	Proxy efficient
Latency overhead	0ms	1-2ms (negligible vs 5-30ms query time)	Acceptable
ECS tasks (bounded pools)	400 conns (23% max) — safe	400 conns + 1-2ms overhead	Skip proxy for ECS

■ Use RDS Proxy for Lambda, skip it for ECS. For ECS tasks with a controlled HikariCP pool (5 services × 4 tasks × pool 20 = 400 connections = 23% of max), the proxy adds latency and complexity for no benefit. Proxy overhead matters at scale: Lambda at 10,000 invocations/second adds 10,000 × 2ms = 20 extra seconds of total latency per second.

06-08

Performance Insights — Metrics & Top SQL

Performance Insights shows Average Active Sessions (AAS) — the number of queries executing or waiting divided by vCPU count. AAS > 1.0 on db.r6g.large (2 vCPUs) means queries are queuing. The Top SQL view ranks queries by DB time, revealing the exact statements causing load.

PI Metric	Healthy Range	Warning	Critical + Required Action
DBLoad (AAS)	< 1.0	1.0 – 2.0	> 2.0 for 5+ min → run EXPLAIN ANALYZE on top SQL
DatabaseConnections	< 800	800 – 1,400	> 1,400 (80% of 1,717) → reduce pool or scale out

PI Metric	Healthy Range	Warning	Critical + Required Action
CommitLatency p95	< 5ms	5ms – 50ms	> 50ms → Aurora storage I/O saturation
ReadLatency p95	< 1ms	1ms – 10ms	> 10ms → BufferCacheHitRatio falling below 99%
ReplicationLag	< 100ms	100ms – 1s	> 1s → remove that reader from reader endpoint
BufferCacheHitRatio	> 99%	95% – 99%	< 95% → scale to larger instance (more RAM)
Deadlocks/min	0	1 – 5	> 5 → fix transaction ordering in application code

pg_stat_statements — Find Worst Queries

```
-- Enable in cluster parameter group:
-- shared_preload_libraries = pg_stat_statements
-- After restart: CREATE EXTENSION pg_stat_statements;
SELECT query,
calls,
ROUND(mean_exec_time::numeric, 2) AS avg_ms,
ROUND(stddev_exec_time::numeric, 2) AS stddev_ms,
ROUND(rows::numeric / NULLIF(calls,0), 1) AS avg_rows,
ROUND(100.0 * shared_blks_hit /
NULLIF(shared_blks_hit + shared_blks_read, 0), 2) AS cache_hit_pct
FROM pg_stat_statements
WHERE calls > 100 -- ignore one-offs
ORDER BY total_exec_time DESC
LIMIT 20;
```

Top SQL Found in ShopFlow PI Review

Query Pattern	PI DBLoad	Before	After	Fix Applied
ProductRepository N+1 (21 SELECTs per page)	Write 4.0+	180ms	8ms	JOIN FETCH @Query — 21q → 1q
OrderRepository Seq Scan (50M rows)	Read 4.0+	40,000ms	0.3ms	Composite idx (user_id, status)
SearchService LIKE '%term%'	ReadLatency 50ms+	850ms	2ms	pg_trgm + GIN on products.name
InventoryService lock waits	CommitLatency spike	Variable	3ms	SELECT FOR UPDATE SKIP LOCKED

■ Performance Insights is free for 7-day retention and is included with Enhanced Monitoring. To enable 2-year retention: \$0.02/vCPU/month = \$0.04/month for r6g.large. Set up a weekly 30-minute PI review as a recurring calendar event. Catching N+1 queries early prevents the 180ms → 8ms improvements from becoming user-visible slowness.

06-09

Query Optimisation — EXPLAIN ANALYZE & Index Strategy

EXPLAIN ANALYZE executes the query and returns real row counts, actual timing, and buffer statistics. BUFFERS option shows "shared hit=X read=Y" — high read vs hit means cache misses from disk. This combination precisely diagnoses missing indexes, bad join order, and insufficient RAM.

EXPLAIN ANALYZE Workflow — Full Template

```
-- IMPORTANT: Wrap DML (UPDATE/DELETE) in transaction to prevent mutation
BEGIN;
EXPLAIN (ANALYZE, BUFFERS, VERBOSE, FORMAT TEXT)
SELECT p.id, p.sku, p.price, p.attributes,
c.name AS category_name
FROM products p
JOIN categories c ON c.id = p.category_id
WHERE p.category_id = $1
AND p.status = 'ACTIVE'
ORDER BY p.created_at DESC
LIMIT 20;
```



```
ROLLBACK; -- safe: EXPLAIN ANALYZE inside transaction rolls back
-- Bad output (missing index):
-- Seq Scan on products (cost=0..189423 rows=450000 width=280)
-- Actual time=4823.2..4911.7 Buffers: shared read=14200 ← disk reads
-- Good output (after CREATE INDEX CONCURRENTLY):
-- Index Scan using idx_products_cat_act on products
-- Actual time=0.3..0.8 Buffers: shared hit=24 read=0 ← all in RAM
```

EXPLAIN Output Term	What It Means	Action Required
Seq Scan on large table	Full table scan — every row read	CREATE INDEX on WHERE/JOIN column
Buffers: shared read=N (large)	N disk block reads — data not in buffer pool	Larger instance (more RAM) or better index
Actual rows >>> Estimated rows	Stale statistics cause wrong plan choice	ANALYZE products; or tune autovacuum
Sort: external merge Disk	Sort spills to temp files (work_mem too low)	Increase work_mem to 16MB in param group
Hash Join on large tables	May be correct — or may be cartesian product	Check estimated vs actual row counts carefully

Index Selection Guide

Access Pattern	Best Index	ShopFlow Example
WHERE col = \$1 (high cardinality)	B-Tree (single column)	WHERE user_id = \$1
WHERE col1 = \$1 AND col2 = \$2	Composite B-Tree (equality first)	WHERE user_id = \$1 AND status = \$2
WHERE jsonb_col @> \$1	GIN on JSONB column	WHERE attributes @> '{"color":"red"}'
WHERE ts_col BETWEEN \$1 AND \$2 (non-app-specific)	B-Tree on timestamp	WHERE created_at BETWEEN \$1 AND \$2
WHERE status = 'ACTIVE' (low cardinality)	Partial B-Tree	WHERE status='ACTIVE' (60% of rows excluded)

■ Never run EXPLAIN ANALYZE on an UPDATE or DELETE statement outside a transaction in production. It executes the statement, modifying rows. Pattern: BEGIN; EXPLAIN ANALYZE DELETE FROM abandoned_carts WHERE ...; ROLLBACK; — the ROLLBACK undoes all mutations while you still see the execution plan with real timing.

06-10 Aurora Security — Network, Encryption & pgaudit

ShopFlow's PCI DSS Level 1 compliance requires four security layers: PRIVATE_ISOLATED subnet (zero internet path to Aurora), KMS CMK with quarterly key rotation, rds.force_ssl=1 rejecting all plaintext connections, and pgaudit logging all DDL and connection events to CloudWatch Logs.

Security Layer	Implementation	ShopFlow Configuration	PCI Requirement
Network isolation	PRIVATE_ISOLATED VPC subnet	no NAT, no internet route to RDS. SG all traffic	Req 11.2.1
Encryption at rest	KMS Customer Managed Key alias/shopflow/aurora-cmk. Auto-rotation enabled	Auto-rotation enabled	Req 3.2.1
TLS in transit	rds.force_ssl=1 cluster param	All non-TLS connections rejected at protocol level	Req 3.2.2
Deletion protection	deletionProtection: true in CDK	CDK destroy blocked. Manual disable in RDS console	Req 3.2.3
Audit logging	pgaudit extension	log: ddl, role, connection. Sent to CloudWatch Logs	Req 10.2.2

Aurora Security CDK

```
// KMS CMK for Aurora encryption at rest
const auroraKey = new kms.Key(this,"AuroraKMSKey",{
  description: "ShopFlow Aurora PostgreSQL 15 CMK",
  alias: "alias/shopflow/aurora-cmk",
  enableKeyRotation: true, // automatic 90-day rotation
  rotationPeriod: Duration.days(90),
  removalPolicy: RemovalPolicy.RETAIN, // NEVER delete this key
});
```

```
// Security group – Aurora only from ECS app layer
const sgRds = new ec2.SecurityGroup(this, "SgRds", {
  vpc, description: "Aurora SG – ECS access only",
  allowAllOutbound: false,
});
sgRds.addIngressRule(
  ec2.Peer.securityGroupId(sgEcsApp.securityGroupId),
  ec2.Port.tcp(5432),
  "Aurora PG – allow from ECS app layer only"
);
```

pgaudit Output — CloudWatch Logs Insights

```
# Sample pgaudit log entry for DDL event:
AUDIT: SESSION,1,1,DDL,CREATE INDEX,,CREATE INDEX CONCURRENTLY
idx_orders_user_st ON orders(user_id,status),<none>
# CloudWatch Logs Insights query – find all schema changes:
fields @timestamp, @message
| filter @message like /AUDIT:.*DDL/
| sort @timestamp desc
| limit 50
```

■ `pgaudit.log="ddl,role,connection"` logs three event types: DDL (CREATE TABLE, ALTER, DROP), role events (GRANT, REVOKE), and connection events (login, logout). This is the minimum for PCI DSS Req 10.2. Adding "read,write" logs every SELECT and DML — useful for forensics but generates very high log volume on production databases.

06-11

IAM Database Auth & Secrets Manager Rotation

IAM database authentication generates short-lived tokens (15-minute TTL) via the AWS SDK Signer. Lambda functions use IAM auth — no static passwords stored. ECS tasks use Secrets Manager with automatic rotation every 90 days via a CloudFormation-managed rotation Lambda function.

IAM Database Authentication — TypeScript Lambda

```
import { Signer } from "@aws-sdk/rds-signer";
// Initialize signer once per Lambda execution context (reused across invocations)
const signer = new Signer({
  region: process.env.AWS_REGION!,
  hostname: process.env.DB_PROXY_ENDPOINT!, // RDS Proxy endpoint
  port: 5432,
  username: "shopflow_lambda", // DB user with rds_iam grant
});
// Token cache – valid for 15 minutes. Refresh 60s early.
let cachedToken: string | null = null;
let tokenExpiry = 0;
export async function getDbToken(): Promise<string> {
  const nowMs = Date.now();
  if (cachedToken && nowMs < tokenExpiry - 60_000)
    return cachedToken; // reuse: safe for 60s after threshold
  cachedToken = await signer.getAuthToken(); // ~50ms HTTPS call to IAM
  tokenExpiry = nowMs + 14 * 60 * 1000; // 14 min (buffer before 15)
  return cachedToken;
}
-- One-time DB setup (run via Secrets Manager admin credential):
CREATE USER shopflow_lambda WITH LOGIN;
GRANT rds_iam TO shopflow_lambda;
GRANT SELECT, INSERT ON orders, order_items TO shopflow_lambda;
```

Auth Method	Credential Type	TTL / Rotation	ShopFlow Usage
IAM DB auth	Signed IAM token (HTTPS)	15 min TTL, auto-refreshed	Lambda functions — no password
Secrets Manager	Username + password JSON	90-day auto-rotation via Lambda	ECS tasks — reference by ARN

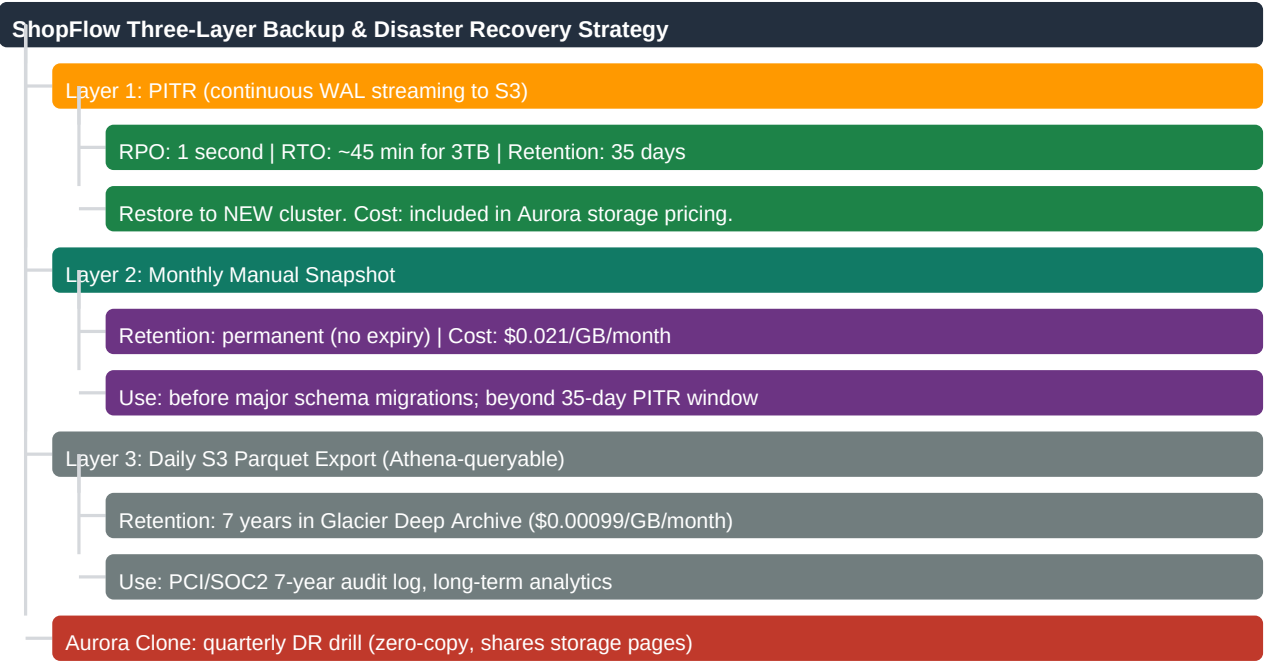
Auth Method	Credential Type	TTL / Rotation	ShopFlow Usage
SSM Parameter Store	Secure string (KMS encrypted)	Manual rotation only	Non-production environments

■ Never cache IAM DB auth tokens across Lambda execution contexts (between warm invocation and cold start). Cached tokens in /tmp or Lambda layers may be stale. Cache WITHIN the execution context only. The signer client itself can be safely reused across invocations — only getToken() needs conditional refresh when within 60s of expiry.

06-12

PITR, Backups & Disaster Recovery

Aurora WAL streams continuously to S3 — PITR restores to any second within 35 days. Restore ALWAYS creates a NEW cluster (no in-place overwrite). Three backup layers cover all recovery scenarios from operational errors (PITR) to 7-year compliance (S3 Parquet export).



PITR Restore Runbook

```
# Step 1: Identify the incident window (UTC)
# - Check CloudWatch Logs for first error timestamp
# - Target: 5 minutes before first error
# Step 2: Restore cluster to target time
aws rds restore-db-cluster-to-point-in-time \
--source-db-cluster-identifier shopflow-db-prod \
--db-cluster-identifier shopflow-recovery-$(date +%Y%m%d-%H%M) \
--restore-type full-copy \
--restore-to-time "2024-11-29T14:30:00Z" \
--vpc-security-group-ids sg-rds-xxxx \
--db-subnet-group-name shopflow-db-subnet
# Step 3: Add writer instance (~30 min)
aws rds create-db-instance \
--db-cluster-identifier shopflow-recovery-$(date +%Y%m%d-%H%M) \
--db-instance-class db.r6g.large --engine aurora-postgresql \
--db-instance-identifier shopflow-recovery-writer
# Step 4: Verify key table row counts, then cut over traffic
```

■ Test PITR quarterly. Quarterly drill: restore cluster → verify 5 key table row counts → measure actual RTO (typically 35-55min for 3TB) → delete restored cluster → update runbook with measured RTO. Most teams discover restore procedure bugs only during a real incident. Find them in a drill.

ShopFlow's schema uses `gen_random_uuid()` PKs (no sequential ID leakage), JSONB for product attributes (flexible catalog without EAV), GIN indexes for `@>` attribute searches, and BRIN indexes on time-series columns — 30x smaller than B-Tree on 50M-row append-only tables.

```
CREATE TABLE products (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sku VARCHAR(100) UNIQUE NOT NULL,
  price NUMERIC(12,2) NOT NULL CHECK (price >= 0),
  attributes JSONB NOT NULL DEFAULT '{}':jsonb,
  status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
  CHECK (status IN ('ACTIVE', 'INACTIVE', 'DRAFT')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- GIN: WHERE attributes @> '{"color":"red","size":"M"}'
CREATE INDEX idx_products_attrs ON products USING GIN(attributes);
-- Partial: ACTIVE rows only (40% of table – 60% excluded)
CREATE INDEX idx_products_cat_act ON products(category_id) WHERE status='ACTIVE';
CREATE TABLE orders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  status VARCHAR(30) NOT NULL DEFAULT 'PENDING'
  CHECK (status IN ('PENDING', 'CONFIRMED', 'SHIPPED', 'DELIVERED')),
  total_amt NUMERIC(12,2) NOT NULL CHECK (total_amt >= 0),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- BRIN: 128 pages on 50M rows vs 4,000+ B-Tree pages = 30x smaller
CREATE INDEX idx_orders_created ON orders USING BRIN(created_at);
-- Composite: order history page – equality + equality
CREATE INDEX idx_orders_user_st ON orders(user_id, status);
```

Index Type	Size on 50M Rows	When to Use	ShopFlow Usage
B-Tree	4,000+ pages	Equality, range, high cardinality	user_id, email, UUID PKs, composites
GIN	8-40K pages (larger!)	JSONB <code>@></code> , full-text tsvector, array overlaps	products.attributes JSONB search
BRIN	128 pages (30x smaller)	Append-only time-series: new rows always later	orders.created_at (50M rows)
Partial B-Tree	Fraction of B-Tree	Status with <code>&lt;</code> 50% selectivity WHERE clause	WHERE status='ACTIVE' only

■ `gen_random_uuid()` prevents sequential ID attacks and competitor intelligence: order ID 1,047,382 in a receipt email reveals your exact order count and growth rate. UUID reveals nothing. Available built-in since PG 13 with no extension. Also prevents API enumeration attacks (GET /orders/1, /orders/2...) that expose private customer data.

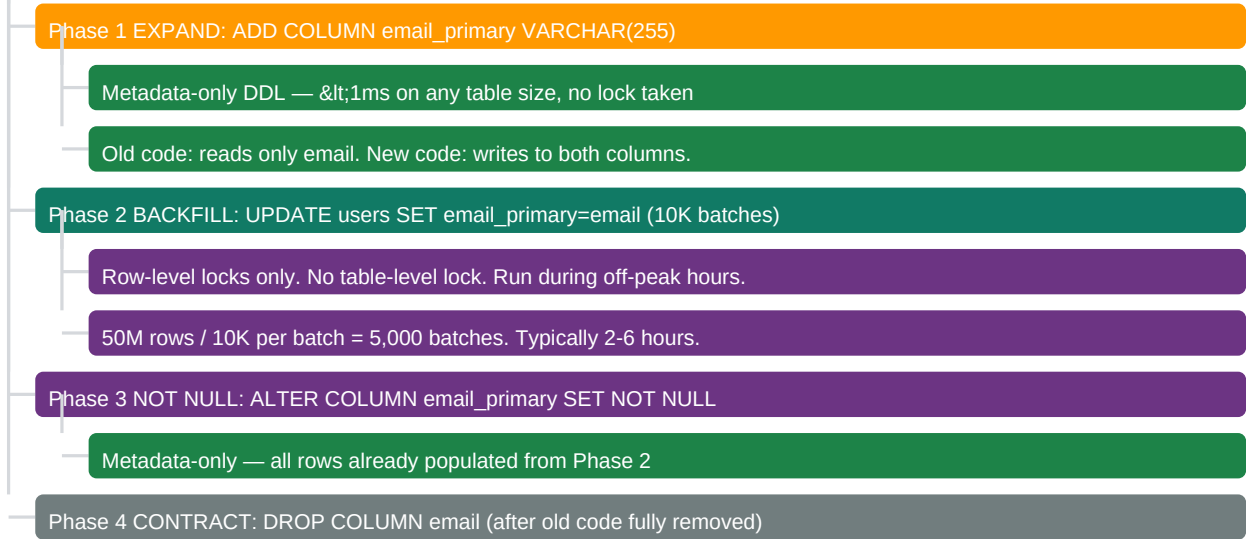
Flyway applies versioned SQL on Spring Boot startup. PostgreSQL DDL is transactional — failed migrations auto-rollback with zero partial state. The expand-contract pattern replaces a 30-minute blocking ALTER TABLE with four atomic metadata-only DDL steps on 50M-row tables.

Flyway Configuration

```
spring:
  flyway:
    enabled: true
    validate-on-migrate: true # checksum verify all applied migrations
    out-of-order: false # strict ordering – no version gaps allowed
    locations: classpath:db/migration
    # Naming: V{version}__{description}.sql (double underscore required!)
    # src/main/resources/db/migration/
    # V1__initial_schema.sql
```

```
# V2__add_orders_table.sql
# V3__add_product_jsonb_attributes.sql
# V4__add_brin_idx_orders_created_at.sql
# R__refresh_product_search_view.sql # Repeatable on checksum change
```

Expand-Contract: users.email → users.email_primary (50M rows)



Method	Duration on 50M Rows	Table Lock?	Service Availability
Single ALTER TABLE RENAME	15-30 minutes	YES — all reads + writes blocked	Service blocked during migration
Phase 1: ADD COLUMN	<1 millisecond	NO — metadata only	No impact, deploy normally
Phase 2: Batched UPDATE	Hours (off-peak schedule)	NO — row-level locks only	No impact, app continues
Phase 3: SET NOT NULL	<1 millisecond	NO — metadata only	No impact, deploy normally
Phase 4: DROP COLUMN	<1 millisecond	NO — metadata only	No impact, all old code removed

■ Never modify an applied Flyway migration file. Flyway stores the SHA-256 checksum of each applied migration in flyway_schema_history. With validate-on-migrate: true, any checksum mismatch causes FlywayValidateException on application startup — all ECS tasks fail to start simultaneously. Create a NEW migration to correct side effects.

06-15 Parameter Groups, Global DB & Cost Optimisation

Aurora cluster parameter groups configure engine-level behavior. Six parameters critical for PCI/SOC2 compliance and performance. Aurora Global Database adds <1-second cross-region replication for EU expansion. Reserved Instances provide 45% discount on compute costs.

Parameter	Default	ShopFlow	Why
rds.force_ssl	0	1	PCI Req 4.1: reject all non-TLS connections
log_min_duration_statement	off	1000ms	Log queries >1s to CloudWatch for PI correlation
work_mem	4 MB	16 MB	Sort + hash joins in memory — no temp file disk I/O
maintenance_work_mem	64 MB	512 MB	CREATE INDEX CONCURRENTLY 8x faster
shared_preload_libraries	—	pg_stat_statements, pgaudit	Query stats for PI + PCI audit trail
pgaudit.log	none	ddl,role,connection	Log schema changes, grants, logins

Aurora Global Database — Phase 2 EU Expansion

PRIMARY: us-east-1 — writer + US reader (existing)

All global writes originate here — single-region writer

SECONDARY: eu-west-1 — read-only cluster (new)

EU catalog reads at 40ms vs 180ms transatlantic

Replication lag: <1s via dedicated Aurora storage channel (not binlog)

Writes from EU routed to us-east-1 via API Gateway + CloudFront

Managed Planned Failover: promotes eu-west-1 in <1min, RPO=1s

Reserved Instance Cost Savings

Instance	On-Demand/hr	1yr RI/hr	Monthly OD	Monthly RI	Saving
Writer: db.r6g.large	\$0.277	\$0.154	\$202	\$112	\$90 (45%)
Reader-1: db.r6g.large	\$0.277	\$0.154	\$202	\$112	\$90 (45%)
Reader-2: db.r6g.medium	\$0.138	\$0.077	\$101	\$56	\$45 (45%)
TOTAL — 3 instances	—	—	\$505/month	\$280/month	\$225/month

■ Changing shared_preload_libraries is a static parameter — requires a DB restart. Aurora rolling restart: modify readers one at a time (no impact on availability), then force writer failover (15-30s), then old writer restarts as reader. Dynamic parameters (work_mem, log_min_duration_statement) take effect on the next new connection — no restart needed.

06-BP

Best Practices & Anti-Patterns

- ✓ Aurora PostgreSQL over RDS: 5x throughput, 15-30s failover, 6-copy storage, 128TB auto-grow
- ✓ Cluster endpoint for writes, reader endpoint for reads — never hardcode instance endpoint DNS
- ✓ promotionTier:1 on SAME instance class as writer — correct sizing after failover (critical)
- ✓ RDS Proxy for Lambda concurrency >100 — 1,000 invocations to 50 stable pool connections
- ✓ HikariCP pool math: $\text{total_tasks} \times \text{maxPool} \leq 80\% \text{ of } \text{max_connections}$ at peak task count
- ✓ HikariCP: maxPool=20, connectionTimeout=30s, leakDetectionThreshold=10s — these three always
- ✓ Performance Insights free 7-day — weekly Top SQL review + EXPLAIN ANALYZE slow queries monthly
- ✓ EXPLAIN (ANALYZE, BUFFERS) — actual timing and buffer hit/read ratio, never estimates alone
- ✓ CREATE INDEX CONCURRENTLY — no table lock, safe on production 50M-row tables during business hours
- ✓ PITR retention to 35 days — PCI DSS Req 10.7, SOC2 CC7.4 compliance requirements
- ✓ Test PITR quarterly — restore → verify 5 tables → measure RTO → delete → update runbook
- ✓ Flyway expand-contract: 4 metadata-only DDL steps vs 30-minute blocking ALTER TABLE
- ✓ JSONB + GIN for product attributes — enables flexible @> search, avoids EAV anti-pattern
- ✓ gen_random_uuid() PKs — hides order count from competitors; prevents sequential API enumeration
- ✓ Aurora Graviton (r6g) — 12% cheaper vs x86 r6 for same vCPU/RAM; consistent with ECS ARM
- ✓ deletionProtection:true in CDK — prevents accidental production cluster deletion in terraform/cdk destroy
- ✓ Monthly manual snapshot before major schema migrations (beyond 35-day PITR window)
- ✓ Cache IAM DB auth tokens in Lambda execution context: 15-min TTL, refresh 60s before expiry
- Never place Aurora in public subnets — PRIVATE_ISOLATED only; no internet route, no NAT

- Never set `maximumPoolSize` without connection math: $\text{tasks} \times \text{pool} > \text{max_connections}$ = random FATALs
- Never send analytics queries to writer endpoint — reader endpoint or S3+Athena export only
- Never store DB passwords in Lambda environment variables — use IAM auth or Secrets Manager ARN
- Aurora PG has NO Backtrack (MySQL only) — PITR always creates a new cluster. RTO ~45 minutes.
- Never edit applied Flyway migration files — checksum mismatch causes all ECS tasks to fail on deploy

06-QR

Quick Reference & Interview Q&A

Concept	Key Value / Rule
Aurora storage copies	6 copies across 3 AZs. Write quorum: 4/6. Read quorum: 3/6. Auto-grows 10GB.
Aurora max storage	128 TB per cluster, automatic, zero-downtime increment.
Failover comparison	Aurora: 15-30s (shared storage). RDS Multi-AZ: 60-120s (EBS promote + DNS).
<code>max_connections</code> formula	$\text{LEAST}(\{\text{DBInstanceClassMemory}/9531392\}, 5000)$. <code>r6g.large</code> (16GB) = 1,717.
HikariCP pool rule	$\text{total_tasks} \times \text{maxPoolSize} \leq 80\%$ of <code>max_connections</code> at all times.
RDS Proxy overhead	1-2ms latency. Use for Lambda > 100 concurrency. Skip for bounded ECS pools.
IAM DB auth token TTL	15 minutes. Cache per Lambda execution context. Refresh 60s before expiry.
PITR max retention	35 days. Always restores to NEW cluster — no in-place. RTO ~45min. RPO 1s.
Aurora Backtrack	MySQL only — NOT available for Aurora PostgreSQL. Use PITR for Aurora PG.
Serverless v2 minimum	0.5 ACU — cannot scale to zero. Always-on ~\$22/month.
BRIN vs B-Tree	BRIN: 128 pages. B-Tree: 4,000+ pages on 50M time-series rows. 30x smaller.
Flyway naming	<code>V{n}__{desc}.sql</code> (double underscore). Never modify applied migration.
Expand-contract summary	ADD COLUMN → batch UPDATE → NOT NULL → DROP old. 4 zero-downtime steps.
<code>pgaudit.log</code> values	<code>none ddl role connection read write</code> . ShopFlow: "ddl,role,connection".
Aurora Global replication	< 1 second via dedicated storage channel. Managed failover: < 1 minute.
RI savings 3-instance cluster	45% discount on 1-year term. ShopFlow: \$225/month saved.
I/O-Optimized breakeven	Switch when I/O charges exceed 25% of total Aurora bill.

Top Interview Questions

Question	Answer
Aurora vs RDS core architecture?	Aurora separates storage from compute. Writer + all readers share one 6-copy distributed storage volume.
When to use RDS Proxy?	Lambda opens 1 connection per invocation. At 1,000 concurrent + 370 ECS = 1,370 connections = 80% of <code>max_connections</code> .
How does PITR work for Aurora PostgreSQL?	Writes to S3 continuously. PITR restores to any second in 35-day window by creating a NEW cluster.
Explain Flyway expand-contract.	4 steps on 50M rows: 1) ADD nullable column — metadata-only, < 1 ms. 2) Batch UPDATE 10K rows. 3) ALTER NOT NULL. 4) DROP old.
HikariCP <code>maximumPoolSize</code> sizing formula?	Formula: $\text{total_tasks} \times \text{maximumPoolSize} \leq 80\%$ of <code>max_connections</code> . Aurora <code>r6g.large</code> : <code>max</code> =1,717. 50% = 858.
Why <code>gen_random_uuid()</code> not BIGSERIAL?	BIGSERIAL order ID 1,047,382 in a receipt email reveals total order count and growth rate — competitor risk.
BRIN vs B-Tree: when each?	B-Tree for random-access equality/range on high cardinality columns (<code>user_id</code> , email, UUID PKs). BRIN for time-series.